API Flask/FastAPI

m2iformation.fr

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

API Flask/FastAPI

Création d'APIs REST Modernes

Utopios® Tous droits réservés

Module 3 - Documentation + Tests + Monitoring

Production-ready avec tests et observabilité

OpenAPI, Swagger, Pytest, Logging, Monitoring et déploiement

Documentation | Tests | Monitoring | Déploiement

Utopios® Tous droits réservés

OpenAPI Specification

Documenter automatiquement vos APIs

OpenAPI (anciennement Swagger) est un standard pour décrire les APIs REST:

- Format YAML ou JSON pour décrire l'API
- Standardisé et reconnu par l'industrie
- FastAPI génère automatiquement OpenAPI depuis vos code
- Compatible avec tous les outils OpenAPI
- Versionage automatique de l'API

Utopios® Tous droits réservés

OpenAPI Specification

Accédez à la spécification:

```
GET http://localhost:8000/openapi.json
```

Avantage: Les clients peuvent générer des SDK automatiquement!

Utopios® Tous droits réservés

Swagger UI et ReDoc

Interfaces interactives pour documenter

FastAPI fournit deux interfaces de documentation:

```
app = FastAPI(
    title="API Utilisateurs",
    description="Gestion complète des utilisateurs",
    version="1.0.0",
    docs_url="/api/docs",              # Swagger UI
    redoc_url="/api/redoc",             # ReDoc
    openapi_url="/api/openapi.json"     # Spec OpenAPI
)
```

Utopios® Tous droits réservés

Swagger UI et ReDoc

Swagger UI (http://localhost:8000/docs):

- Interface interactive et conviviale
- Testez directement les endpoints
- Essayez les paramètres

ReDoc (http://localhost:8000/redoc):

- Documentation épurée et lisible
- Parfait pour la lecture

Utopios® Tous droits réservés

Documenter les Endpoints

Descriptions et exemples détaillés

Documentez vos endpoints avec des descriptions riches:

```python
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class User(BaseModel):
    """Modèle utilisateur"""
    id: int = Field(..., description="ID unique")
    name: str = Field(..., description="Nom complet")
    email: str = Field(..., description="Adresse email")
    age: int = Field(..., ge=0, le=150, description="Age")

@app.get("/users/{user_id}", tags=["Users"], summary="Récupérer un utilisateur")
async def get_user(
    user_id: int = Field(..., description="ID de l'utilisateur")
) -> User:
    """
    Récupère les détails d'un utilisateur spécifique.

    - **user_id**: L'ID de l'utilisateur à récupérer
    - Retourne: Les détails complets de l'utilisateur
    """
    return User(id=user_id, name="John", email="john@example.com", age=30)
```

Utopios® Tous droits réservés

Schemas OpenAPI Personnalisés

Exemples et métadonnées avancées

Ajoutez des exemples et métadonnées dans OpenAPI:

```python
from pydantic import BaseModel, Field

class Product(BaseModel):
    """Produit dans l'inventaire"""
    name: str = Field(
        ...,
        title="Nom du produit",
        description="Nom unique du produit",
        example="Laptop"
    )
    price: float = Field(
        ...,
        gt=0,
        title="Prix",
        description="Prix en USD",
        example=999.99
    )
    quantity: int = Field(
        default=0,
        title="Quantité",
        description="Nombre d'articles en stock",
        example=5
    )

    model_config = {
        "json_schema_extra": {
            "example": {
                "name": "MacBook Pro",
                "price": 1499.99,
                "quantity": 10
            }
        }
    }
```

Utopios® Tous droits réservés

Introduction à Pytest

Framework de test puissant et flexible

Pytest est le framework de test standard pour Python:

- Syntaxe simple et pythonique
- Découverte automatique des tests
- Fixtures pour réutiliser le code
- Parameterization pour tester plusieurs cas
- Plugins extendus (cov, asyncio, etc)

Utopios® Tous droits réservés

Introduction à Pytest

Installez Pytest:

```
pip install pytest pytest-asyncio pytest-cov httpx
```

Créez votre premier test:

```
def test_sum():
    assert 2 + 2 == 4

# Lancez:
pytest test_file.py
```

Utopios® Tous droits réservés

Test Client FastAPI

Tester vos endpoints facilement

Utilisez TestClient pour tester vos endpoints:

```python
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_read_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello"}

def test_create_user():
    response = client.post("/users", json={
        "name": "John",
        "email": "john@example.com",
        "age": 30
    })
    assert response.status_code == 201
    assert response.json()["id"] == 1

def test_get_user():
    response = client.get("/users/1")
    assert response.status_code == 200
    assert response.json()["name"] == "John"
```

Utopios® Tous droits réservés

Unit Tests vs Integration Tests

Deux approches complémentaires

Unit Tests: Tester une fonction isolée

```
def validate_email(email: str) -> bool:
    return "@" in email and "." in email

def test_validate_email():
    assert validate_email("test@example.com") == True
    assert validate_email("invalid") == False
```

Utopios® Tous droits réservés

Unit Tests vs Integration Tests

Deux approches complémentaires

Integration Tests: Tester le flux complet

```python
def test_user_lifecycle():
    # 1. Créer un utilisateur
    create_response = client.post("/users", json={...})
    user_id = create_response.json()["id"]

    # 2. Récupérer l'utilisateur
    get_response = client.get(f"/users/{user_id}")
    assert get_response.status_code == 200

    # 3. Mettre à jour
    update_response = client.put(f"/users/{user_id}", json={...})
    assert update_response.status_code == 200

    # 4. Supprimer
    delete_response = client.delete(f"/users/{user_id}")
    assert delete_response.status_code == 204
```

Utopios® Tous droits réservés

Fixtures et Factories

Réutiliser le code de test

Utilisez les fixtures pour créer les données de test:

```python
import pytest
from fastapi.testclient import TestClient

@pytest.fixture
def client():
    """Fixture: client de test"""
    return TestClient(app)

@pytest.fixture
def sample_user():
    """Fixture: utilisateur exemple"""
    return {
        "name": "John",
        "email": "john@example.com",
        "age": 30
    }

def test_create_user(client, sample_user):
    response = client.post("/users", json=sample_user)
    assert response.status_code == 201

def test_create_multiple(client, sample_user):
    # Réutilise la fixture dans un autre test
    for i in range(5):
        response = client.post("/users", json={
            **sample_user,
            "name": f"User {i}"
        })
        assert response.status_code == 201
```

Utopios® Tous droits réservés

Mocking et Patches

Isoler les dépendances externes

Mockez les dépendances pour tester en isolation:

```python
from unittest.mock import patch, MagicMock
import pytest

@pytest.fixture
def mock_db():
    """Mock la base de données"""
    return MagicMock()

def test_get_user_with_mock(mock_db):
    # Configurer le mock
    mock_db.query.return_value.filter.return_value.first.return_value = {
        "id": 1,
        "name": "John"
    }

    # Patcher la dépendance
    with patch('main.get_db', return_value=mock_db):
        response = client.get("/users/1")
        assert response.status_code == 200
        assert response.json()["name"] == "John"

def test_send_email_mock():
    with patch('main.send_email') as mock_send:
        response = client.post("/register", json={...})
        # Vérifier que send_email a été appelé
        assert mock_send.called
```

Utopios® Tous droits réservés

Couverture de Code avec Pytest-Cov

Mesurer la qualité de vos tests

Mesurez la couverture de votre code:

```
pip install pytest-cov

# Lancer avec rapport de couverture
pytest --cov=main --cov-report=html

# Résultats:
# main.py 85% (17 of 20 lignes couvertes)
```

- Identifiez les portions non testées
- Fixez un seuil minimum (ex: 80%)
- Générez des rapports HTML
- Intégrez dans CI/CD

Objectif: Couverture > 80% pour la production

Utopios® Tous droits réservés

Logging Setup

Tracer vos application en production

Configurez le logging structuré:

```python
import logging
from logging.config import dictConfig

logging_config = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "default": {
            "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s",
        },
    },
    "handlers": {
        "default": {
            "formatter": "default",
            "class": "logging.StreamHandler",
            "stream": "ext://sys.stderr",
        },
        "file": {
            "formatter": "default",
            "class": "logging.FileHandler",
            "filename": "app.log",
        },
    },
    "loggers": {
        "app": {"handlers": ["default", "file"], "level": "INFO"},
    },
}

dictConfig(logging_config)
logger = logging.getLogger("app")

@app.get("/users/{user_id}")
async def get_user(user_id: int):
    logger.info(f"Récupération utilisateur {user_id}")
    return {"id": user_id}
```

Utopios® Tous droits réservés

Prometheus Metrics

Monitorer les performances

Exposez les métriques Prometheus:

```
from prometheus_client import Counter, Histogram, generate_latest
from fastapi.responses import Response
import time

request_count = Counter('requests_total', 'Total requests', ['method', 'endpoint'])
request_duration = Histogram('request_duration_seconds', 'Request duration', ['method'])

@app.middleware("http")
async def add_metrics(request, call_next):
    start_time = time.time()

    response = await call_next(request)

    # Enregistrer les métriques
    request_count.labels(
        method=request.method,
        endpoint=request.url.path
    ).inc()

    duration = time.time() - start_time
    request_duration.labels(method=request.method).observe(duration)

    return response

@app.get("/metrics")
async def metrics():
    return Response(generate_latest(), media_type="text/plain")
```

Utopios® Tous droits réservés

Health Checks et Observabilité

Monitoring dans Kubernetes

Implémentez les health checks:

```python
@app.get("/health", tags=["Health"])
async def health_check():
    """Liveness probe"""
    return {"status": "alive"}

@app.get("/ready", tags=["Health"])
async def readiness_check():
    """Readiness probe"""
    try:
        # Vérifier la base de données
        db.query("SELECT 1")
        # Vérifier les dépendances
        cache.ping()
        return {"status": "ready"}
    except Exception as e:
        return {"status": "not ready", "error": str(e)}, 503

@app.get("/info", tags=["Health"])
async def info():
    """Informations sur l'API"""
    return {
        "version": "1.0.0",
        "environment": "production",
        "commit": "abc123",
        "uptime": get_uptime()
    }
```

Utopios® Tous droits réservés

Docker et Docker Compose

Containeriser votre API

Créez un Dockerfile pour votre API:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Utopios® Tous droits réservés

Docker et Docker Compose

Déployez avec Docker Compose:

```yaml
version: '3.8'
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://user:pass@db:5432/app
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: password
```

Lancez:

```
docker-compose up
```

Utopios® Tous droits réservés

Environment Variables et Configuration

Gérer les configurations en production

Utilisez les variables d'environnement:

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    app_name: str = "Mon API"
    database_url: str = "sqlite:///./test.db"
    secret_key: str = "your-secret-key"
    debug: bool = False
    log_level: str = "INFO"

    class Config:
        env_file = ".env"

settings = Settings()

# Utilisation
app = FastAPI(title=settings.app_name)

@app.on_event("startup")
async def startup():
    logger.setLevel(settings.log_level)
    logger.info(f"Application démarrée en {'debug' if settings.debug else 'production'}")
```

Utopios® Tous droits réservés

Environment Variables et Configuration

Créez un fichier .env:

```
DATABASE_URL=postgresql://user:pass@localhost/app
SECRET_KEY=my-secret-key-prod
DEBUG=false
LOG_LEVEL=WARNING
```

Utopios® Tous droits réservés

Déploiement en Production

Passer de développement à production

- **Disable debug mode:** debug=False
- **Use production ASGI server:** Uvicorn + Gunicorn/Hypercorn
- **HTTPS/TLS:** Certificates, reverse proxy (Nginx)
- **Database:** PostgreSQL, migrations Alembic
- **Monitoring:** Prometheus, Grafana, Sentry
- **Logging:** ELK Stack, Loki, Splunk
- **CI/CD:** GitHub Actions, GitLab CI, Jenkins

Utopios® Tous droits réservés

Déploiement en Production

```
# Production avec Gunicorn
pip install gunicorn

gunicorn -w 4 -k uvicorn.workers.UvicornWorker main:app
```

Tip: Utilisez Kubernetes pour orchestrer les conteneurs!

Utopios® Tous droits réservés

Démos et Tests Complets

Voir tout ensemble

Démonstration complète de production:

```
# Exécuter l'API complète avec monitoring
uvicorn demos/demo7_complete_api:app --reload

# Visiter la documentation
http://localhost:8000/docs

# Voir les métriques
http://localhost:8000/metrics

# Exécuter les tests
pytest tests/ -v --cov=app

# Voir le rapport de couverture
open htmlcov/index.html
```

Utopios® Tous droits réservés

Démos et Tests Complets

- Tests unitaires et d'intégration
- Documentation complète OpenAPI
- Monitoring avec Prometheus
- Logging structuré

Ressources: https://docs.pytest.org/ et https://prometheus.io/

Utopios® Tous droits réservés

Résumé - Module 3

- **OpenAPI:** Documenter automatiquement
- **Swagger UI:** Interface interactive
- **Pytest:** Framework de test puissant
- **Unit & Integration Tests:** Couvrir tous les cas
- **Mocking:** Isoler les dépendances
- **Couverture de code:** Qualité des tests
- **Logging:** Tracer l'application
- **Monitoring:** Prometheus, Health checks
- **Docker:** Containeriser et déployer

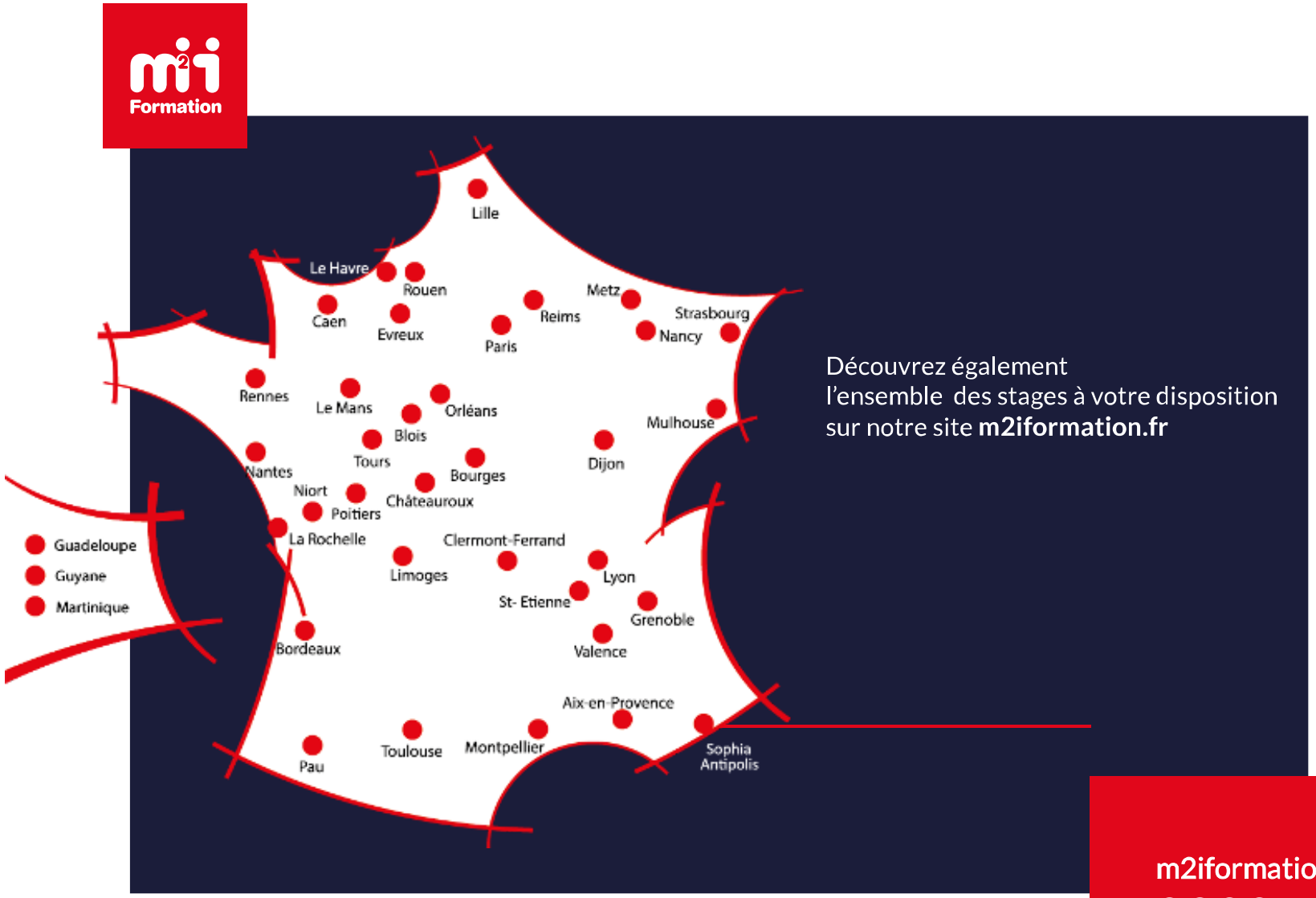
Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Merci pour votre attention

Des questions ?

Utopios® Tous droits réservés



Découvrez également
l'ensemble des stages à votre disposition
sur notre site **m2information.fr**

m2iformation.fr